Grenoble LTS: fresh build 2017–2022 (bike LTS 1–4) + 2026 (bike LTS 1–4 + car LTS 5)

This notebook ignores every earlier cache and pickle. Everything is downloaded again into a **new run folder**.

- **2017–2022:** historical OSM snapshot (31 Dec of each year), bicycle network, LTS 1–4.
- **2026:** current OSM. Bicycle network LTS 1–4, **plus** car roads that are not in the bicycle network as **LTS 5**. An LTS 5 road is flagged `mandatory_upgrade_to_lts1=True` when it is dangerous (speed ≥ 50 km/h, ≥ 4 lanes, or primary/secondary class) and cycling is not prohibited on it (motorway/trunk/ `bicycle=no` get `bicycle_prohibited=True` instead).
- **Simplification (every year):** single undirected `nx.Graph` (worst LTS kept on parallel edges), largest connected component. A degree-2 junction is removed when both edges have the same LTS; the new edge length is the **sum** of the two. For LTS 5, the merged edge is flagged for an upgrade to LTS 1 if **at least one** of the two edges was flagged.
- **Metrics** are computed on the simplified graph: cycling length and largest-connected-component length at LTS ≤ 1, 2, 3, 4. **LTS 5 is never counted** (it is only a candidate for future upgrades).
- The simplified 2026 graph (bikes + cars) used for the metrics is saved as a pickle.

Files needed next to this notebook: `overpass_robust.py`, `grenoble_lts_graph.py`, `grenoble_workflow_fixed.py`, `grenoble_2026_extension.py`, `grenoble_lts_historical.py`, `grenoble_simplification_exact.py`, `grenoble_metrics_exponential.py`, `grenoble_polygon.wkb`.

1. Settings

In [1]:
```
from pathlib import Path
import pickle
import networkx as nx
import pandas as pd
from shapely import wkb
from IPython.display import display

from overpass_robust import download_graph, OVERPASS_ENDPOINTS
from grenoble_lts_graph import configure_osmnx, annotate_graph,
largest_weak_component, normalize_polygon, _values
from grenoble_lts_historical import normalize_snapshot_date
from grenoble_workflow_fixed import prepare_bike
from grenoble_2026_extension import _missing_car_network
from grenoble_simplification_exact import simplify_lts_graph, check_graph,
plot_topology
from grenoble_metrics_exponential import annual_metrics, exponential_weights

# A NEW run name = a completely fresh build (new download cache, new
outputs).
# Keep the same name to resume a run that stopped halfway (finished years
are not downloaded again).
RUN_NAME = "fresh_2026-10-21"
REBUILD = False        # True = also ignore pickles inside THIS run folder
and redo every year

HISTORICAL_YEARS = list(range(2017, 2023))
CURRENT_YEAR = 2026
TIMEOUT = 600          # seconds the Overpass server may work on one query
MATCH = "way"          # "way": car road is LTS 5 only if its OSM way is not
in the bike graph
                       # "endpoints": original rule (can duplicate streets
the bike graph split differently)
ENDPOINTS = OVERPASS_ENDPOINTS   # tried in order if one times out

RUN = Path("grenoble_lts_output") / RUN_NAME
CACHE = RUN / "osm_http_cache"
RUN.mkdir(parents=True, exist_ok=True)

with open("grenoble_polygon.wkb", "rb") as f:
    poly = wkb.loads(f.read())
polygon = normalize_polygon(poly)
configure_osmnx()   # keeps cycleway/maxspeed/lanes/bicycle tags needed by
the classifier
print("Polygon valid:", poly.is_valid, "| Output:", RUN.resolve())
```

```
Polygon valid: True | Output: C:\Users\P323041\OneDrive - University of
Groningen\Documents\Important\Poyosky\21-10-
2026\Bebote_cloud\grenoble_lts_output\fresh_2026-10-21
```

2. Build functions

In [2]:

```
def save(obj, path):
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("wb") as f:
        pickle.dump(obj, f, protocol=pickle.HIGHEST_PROTOCOL)

def load(path):
    with path.open("rb") as f:   # only open your own pickle files
        return pickle.load(f)

def snapshot(year):
    """Historical years → 31 Dec timestamp; 2026 → None (current OSM)."""
    return None if year == CURRENT_YEAR else
normalize_snapshot_date(str(year))

def get_bike(year):
    """Download the bicycle network, classify raw LTS, map to LTS 1-4."""
    path = RUN / str(year) / f"grenoble_bike_{year}_prepared.pkl"
    if path.exists() and not REBUILD:
        return load(path)
    date = snapshot(year)
    G = download_graph(polygon, "bike", date=date, server_timeout=TIMEOUT,
endpoints=ENDPOINTS,
                       cache_folder=CACHE / (date[:10] if date else
"current"))
    steps = [(u, v, k) for u, v, k, d in G.edges(keys=True, data=True) if
"steps" in _values(d, "highway")]
    G.remove_edges_from(steps)
    G.remove_nodes_from(list(nx.isolates(G)))
    G = annotate_graph(G)                       # raw lts_score
0,1,2,2.5,3,4,5
    bike = prepare_bike(largest_weak_component(G))   # → key/lts 1..4
    bike.graph["year"] = year
    save(bike, path)
    return bike

def get_cars(year):
    path = RUN / str(year) / f"grenoble_car_{year}_raw.pkl"
    if path.exists() and not REBUILD:
        return load(path)
    date = snapshot(year)
    cars = download_graph(polygon, "drive", date=date,
server_timeout=TIMEOUT, endpoints=ENDPOINTS,
                          cache_folder=CACHE / (date[:10] if date else
"current"))
    save(cars, path)
    return cars

def build_year(year):
    out = RUN / str(year) / f"grenoble_lts_{year}_SIMPLE.pkl"
    if out.exists() and not REBUILD:
```

```
        print(f"[{year}] loaded {out}")
        return load(out)
    bike = get_bike(year)
    if year == CURRENT_YEAR:
        source = _missing_car_network(bike, get_cars(year), match=MATCH)   #
LTS 1-4 + LTS 5
        save(source, RUN / str(year) /
f"grenoble_lts5_{year}_pre_simplification.pkl")
    else:
        source = bike
    H = simplify_lts_graph(source)   # same-LTS degree-2 removal, summed
length, OR of upgrade flag
    H.graph.update(year=year, city="Grenoble", snapshot=snapshot(year) or
"current OSM")
    check = check_graph(H)
    assert type(H) is nx.Graph and check["connected"]
    assert check["remaining_contractible_degree2_nodes"] == 0
    levels = {d["key"] for _, _, d in H.edges(data=True)}
    if year == CURRENT_YEAR:
        assert levels ⩽ {1, 2, 3, 4, 5} and 5 in levels, "No LTS 5 roads in
2026"
    else:
        assert levels ⩽ {1, 2, 3, 4}
    save(H, out)
    print(f"[{year}] saved {out}")
    return H
```

3. Build all seven networks

Each year is saved as soon as it finishes. If a year fails, rerun this cell: finished years load from disk.

In [3]:
```
metric_graphs, failed = {}, {}
for year in HISTORICAL_YEARS + [CURRENT_YEAR]:
    try:
        metric_graphs[year] = build_year(year)
    except Exception as exc:
        failed[year] = repr(exc)
        print(f"[{year}] FAILED: {exc}")
print("Done:", sorted(metric_graphs), "| Failed:", failed)
```

```
[2017] loaded grenoble_lts_output\fresh_2026-10-
21\2017\grenoble_lts_2017_SIMPLE.pkl
[2018] loaded grenoble_lts_output\fresh_2026-10-
21\2018\grenoble_lts_2018_SIMPLE.pkl
[2019] loaded grenoble_lts_output\fresh_2026-10-
21\2019\grenoble_lts_2019_SIMPLE.pkl
[2020] loaded grenoble_lts_output\fresh_2026-10-
21\2020\grenoble_lts_2020_SIMPLE.pkl
[2021] loaded grenoble_lts_output\fresh_2026-10-
21\2021\grenoble_lts_2021_SIMPLE.pkl
[2022] loaded grenoble_lts_output\fresh_2026-10-
21\2022\grenoble_lts_2022_SIMPLE.pkl
[2026] loaded grenoble_lts_output\fresh_2026-10-
21\2026\grenoble_lts_2026_SIMPLE.pkl
Done: [2017, 2018, 2019, 2020, 2021, 2022, 2026] | Failed: {}
```

4. Metrics on the simplified graphs (LTS 5 excluded)

Length = sum of LTS 1–4 edge lengths. LCC_t = length of the largest connected component (by nodes) using only edges with LTS ≤ t. The normalized values divide by a fixed reference (default: the largest cycling length of the seven years).

In [4]:
```
if failed:
    raise RuntimeError(f"Rerun section 3 first; missing years:
{sorted(failed)}")

KAPPA = 1.0
metrics = annual_metrics(metric_graphs, kappa=KAPPA,
reference_length_km=None)
cols = ["year", "cycling_length_km", "normalized_network_length",
        "lcc_lts1_km", "lcc_lts2_km", "lcc_lts3_km", "lcc_lts4_km",
        "weighted_lcc_normalized", "lts5_edges", "lts5_mandatory_edges"]
display(metrics[cols].round(4))
print("Weights:", exponential_weights(KAPPA))

csv_path = RUN / "grenoble_2017_2022_2026_SIMPLIFIED_metrics.csv"
txt_path = RUN / "grenoble_2017_2022_2026_SIMPLIFIED_metrics.txt"
metrics.to_csv(csv_path, index=False)
with txt_path.open("w", encoding="utf-8") as f:
    f.write("Grenoble simplified cycling metrics, fresh build " + RUN_NAME +
"\n")
    f.write("2017-2022: bicycle LTS1-4 simplified graphs (historical OSM, 31
Dec)\n")
    f.write("2026: bicycle LTS1-4 + missing car roads LTS5 simplified
together; LTS5 excluded from metrics\n")
    f.write(f"Car matching rule: {MATCH}; kappa={KAPPA}\n\n")
    f.write(metrics.to_string(index=False, float_format=lambda x: f"
{x:.6f}") + "\n")
print("Saved:", csv_path, "and", txt_path)
```

Columns 1–8 of 11

	year	cycling_length_km	normalized_network_length	lcc_lts1_km	lcc_lts2_km	lcc_lts3_km	lcc_lts4_km
0	2017	389.0346	0.9095	15.3533	335.2093	385.0795	389.0346
1	2018	392.6995	0.9181	15.3789	338.0646	388.7444	392.6995
2	2019	403.6980	0.9438	15.9680	346.7153	400.0826	403.6980
3	2020	405.0395	0.9469	29.1247	351.4929	402.0220	405.0395
4	2021	413.4887	0.9667	32.6927	361.2714	410.5041	413.4887
5	2022	415.4487	0.9713	31.0489	368.6580	413.1105	415.4487
6	2026	427.7461	1.0000	53.6389	392.1423	425.3733	427.7461

Columns 9–11 of 11 · first column repeated

	weighted_lcc_normalized	lts5_edges	lts5_mandatory_edges
0	0.3164	0	0
1	0.3190	0	0
2	0.3278	0	0
3	0.3508	0	0

	weighted_lcc_normalized	lts5_edges	lts5_mandatory_edges
4	0.3639	0	0
5	0.3662	0	0
6	0.4166	80	4

```
Weights: {1: 0.6439142598879724, 2: 0.23688281808991013, 3:
0.08714431874203257, 4: 0.03205860328008499}
Saved: grenoble_lts_output\fresh_2026-10-
21\grenoble_2017_2022_2026_SIMPLIFIED_metrics.csv and
grenoble_lts_output\fresh_2026-10-
21\grenoble_2017_2022_2026_SIMPLIFIED_metrics.txt
```

5. The saved 2026 graph (bikes + cars) used for the metrics

In [5]:
```
path_2026 = RUN / str(CURRENT_YEAR) /
f"grenoble_lts_{CURRENT_YEAR}_SIMPLE.pkl"
G_2026 = load(path_2026)
assert nx.utils.graphs_equal(G_2026, metric_graphs[CURRENT_YEAR])   # same
graph the metrics used

lts5 = [d for _, _, d in G_2026.edges(data=True) if d["key"] == 5]
print("File:", path_2026.resolve())
print(f"{G_2026.number_of_nodes():,} nodes, {G_2026.number_of_edges():,}
edges")
print("Edges per LTS:", dict(sorted(check_graph(G_2026)
["lts_counts"].items())))
print(f"LTS 5 edges: {len(lts5):,} | must upgrade to LTS 1:
{sum(d['mandatory_upgrade_to_lts1'] for d in lts5):,}"
      f" | cycling prohibited: {sum(bool(d.get('bicycle_prohibited')) for d
in lts5):,}")
print(f"LTS 5 length: {sum(d['distance'] for d in lts5)/1000:.2f} km, of
which mandatory "
      f"{sum(d['distance'] for d in lts5 if
d['mandatory_upgrade_to_lts1'])/1000:.2f} km")
```

```
File: C:\Users\P323041\OneDrive - University of
Groningen\Documents\Important\Poyosky\21-10-
2026\Bebote_cloud\grenoble_lts_output\fresh_2026-10-
21\2026\grenoble_lts_2026_SIMPLE.pkl
5,829 nodes, 8,022 edges
{'nodes': 5829, 'edges': 8022, 'multigraph': False, 'connected': True,
'remaining_contractible_degree2_nodes': 0, 'lts_counts': {2: 3380, 1: 4069,
3: 472, 5: 80, 4: 21}, 'mandatory_lts5_edges': 4}
Edges per LTS: {1: 4069, 2: 3380, 3: 472, 4: 21, 5: 80}
LTS 5 edges: 80 | must upgrade to LTS 1: 4 | cycling prohibited: 69
LTS 5 length: 32.05 km, of which mandatory 1.05 km
```

6. Plots of the simplified networks

Straight lines between the remaining nodes (topology view). Grey = LTS 5, purple = LTS 5 that must be upgraded to LTS 1.

In [6]:
```
for year, G in sorted(metric_graphs.items()):
    print(f"{year}: {G.number_of_nodes():,} nodes, {G.number_of_edges():,} edges")
    fig = plot_topology(G, show_nodes=False)
```

```
2017: 4,501 nodes, 6,469 edges
```

Interactive output (application/vnd.plotly.v1+json) is not included in the PDF.

```
2018: 4,593 nodes, 6,569 edges
```

Interactive output (application/vnd.plotly.v1+json) is not included in the PDF.

```
2019: 4,767 nodes, 6,802 edges
```

Interactive output (application/vnd.plotly.v1+json) is not included in the PDF.

```
2020: 4,841 nodes, 6,877 edges
```

Interactive output (application/vnd.plotly.v1+json) is not included in the PDF.

```
2021: 5,033 nodes, 7,103 edges
```

Interactive output (application/vnd.plotly.v1+json) is not included in the PDF.

```
2022: 5,168 nodes, 7,183 edges
```

Interactive output (application/vnd.plotly.v1+json) is not included in the PDF.

```
2026: 5,829 nodes, 8,022 edges
```

Interactive output (application/vnd.plotly.v1+json) is not included in the PDF.